

Behavioral Detection & Alert Rules

What makes a behavior rule

A rule = a pattern over telemetry (process, network, registry) describing an attack *shape*, not a file hash.

Sigma: the portable detection language

```
title: Suspicious Office -> PowerShell Child Process
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    ParentImage|endswith:
      - '\winword.exe'
      - '\excel.exe'
      - '\outlook.exe'
    Image|endswith: '\powershell.exe'
  filter:
    Image|startswith: 'C:\Windows\System32\'
  condition: selection and not filter
fields:
  - ParentImage
  - Image
level: high
```

Toolchain: Sigmahqs (search engine), sigma-cli converts to Splunk/ES/Elastic from the rule.

Classic rule ideas (LOLBAS chains)

Office → (word/excel/outlook) → powershell/cmd/mshta
powershell → network (any) with -enc/-e argument
regsvr32 with remote URL (scrobj abuse)

LSASS read access (handle) from low-priv process
msbuild/wscript with suspicious child or remote script

Tuning: kill the noise, keep the signal

Filter known-safe: signed updaters, corporate inventory (baseline)

Stack context: only alert on 'spawned by office' combos as high-severity

Time window correlation (network + process within seconds)

Test a rule against 2 weeks of real data BEFORE enabling – measure FP rate

Set severity by damage potential, not alert volume

The metacognition

Every rule encodes a hypothesis about an adversary's move. When it FPs, the hypothesis was too broad; tighten. When it misses, add a second path; correlate.

A detection library's real value is proportional to its noise-to-signal ratio. Rule-writing is 30% security understanding, 70% data discipline.

[← Telemetry Sources: OS Events...](#)

[Threat Hunting: Hypothesis-Dri...](#)